

Programming 2025/2026

Laboratory

Session 1 (April 14)

Introduction

First Cycle Degree/Bachelor in Genomics
Dept. Of Pharmacy and Biotechnology (FaBiT)
University of Bologna



Course Info

- Teaching staff from dep. of Computer Science and Engineering (DISI)
 - Zeynep KIZILTAN (lecturer, associate professor in AI)
 - Email: zeynep.kiziltan@unibo.it
 - Nicolò Donati (tutor, Ph.D. student)
 - Email: n.donati@unibo.it
 - Appointment at Teams upon request by email
- Learning outcome
 - Practical **programming** experience **using Python**
 - **THIS IS NOT A PYTHON COURSE!** Rather, a programming course that uses Python as a tool

Course Info

- Lab sessions
 - 15 sessions of 4 hours
 - Timetable
 - Usually on Tuesdays and Fridays, with some exceptions
 - Check <https://corsi.unibo.it/1cycle/Genomics/timetable>
 - Aim at finishing the topics by May 21
 - Remaining sessions: exam-like questions

Course Info

- How do we conduct a lab session?
 - Recall of some topics covered during the lectures of Prof. Amoroso
 - Exercises on the topics
 - First the students give it a try
 - Can work as a **group** of two **quietly**
 - **Interaction with the teacher and the tutor** is important
 - Clarification of the exercise
 - Problems with the code, etc
 - Later a possible solution is explained
 - Increase in exercise complexity over time

Course Info

- Homework exercises
 - Not mandatory but recommended for a proper preparation
 - Remember you need lots of practice!
 - Selected exercises will be solved during the lab sessions
 - Some **extra online sessions** with the tutor
 - ~~Monday April 20~~ Thursday April 23 (10-12 am)
 - Thursday May 07 pm (10-12 am)

Course Info

- Material and communication
 - Course Virtuale page (sign up if you haven't done it already)
 - Check your UniBo email for announcements!
- Exam
 - One unique exam together with prof. Amoroso
 - Composed of a programming part (using the lab computers) and an oral part on the same day
 - For exam dates and registration, follow <https://corsi.unibo.it/1cycle/Genomics/exam-dates>
 - The first four dates are fixed to June 17, July 8, July 17, Sept 9
 - Two additional during January and February

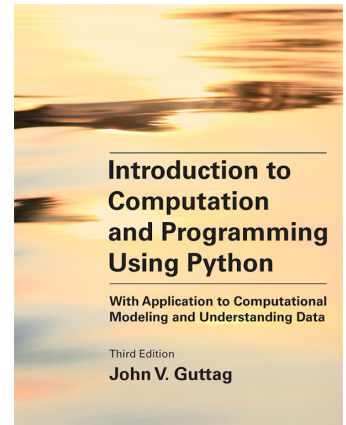
Course Info

- How to prepare for the exam?
- Interactive attendance to the lectures of prof. Amoroso



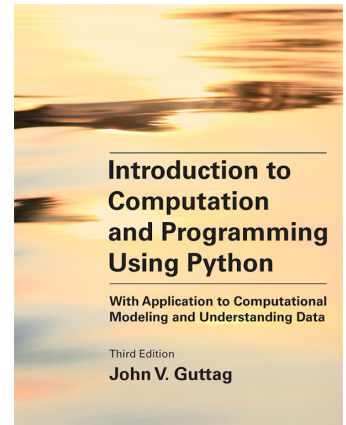
Course Info

- How to prepare for the exam?
 - Interactive attendance to the lectures of prof. Amoroso
 - Self-study of the textbook



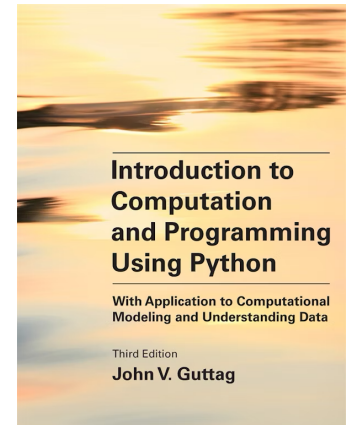
Course Info

- How to prepare for the exam?
 - Interactive attendance to the lectures of prof. Amoroso
 - Self-study of the textbook
 - Interactive attendance to the lab sessions



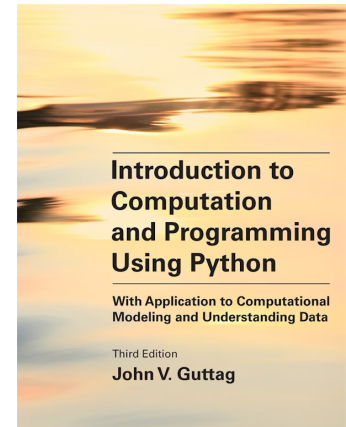
Course Info

- How to prepare for the exam?
 - Interactive attendance to the lectures of prof. Amoroso
 - Self-study of the textbook
 - Interactive attendance to the lab sessions
 - Self-practice at home with homework exercises



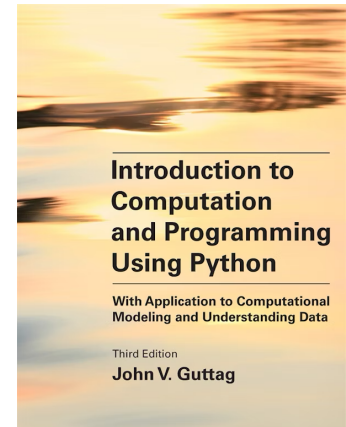
Course Info

- How to prepare for the exam?
 - Interactive attendance to the lectures of prof. Amoroso
 - Self-study of the textbook
 - Interactive attendance to the lab sessions
 - Self-practice at home with homework exercises
 - Group working



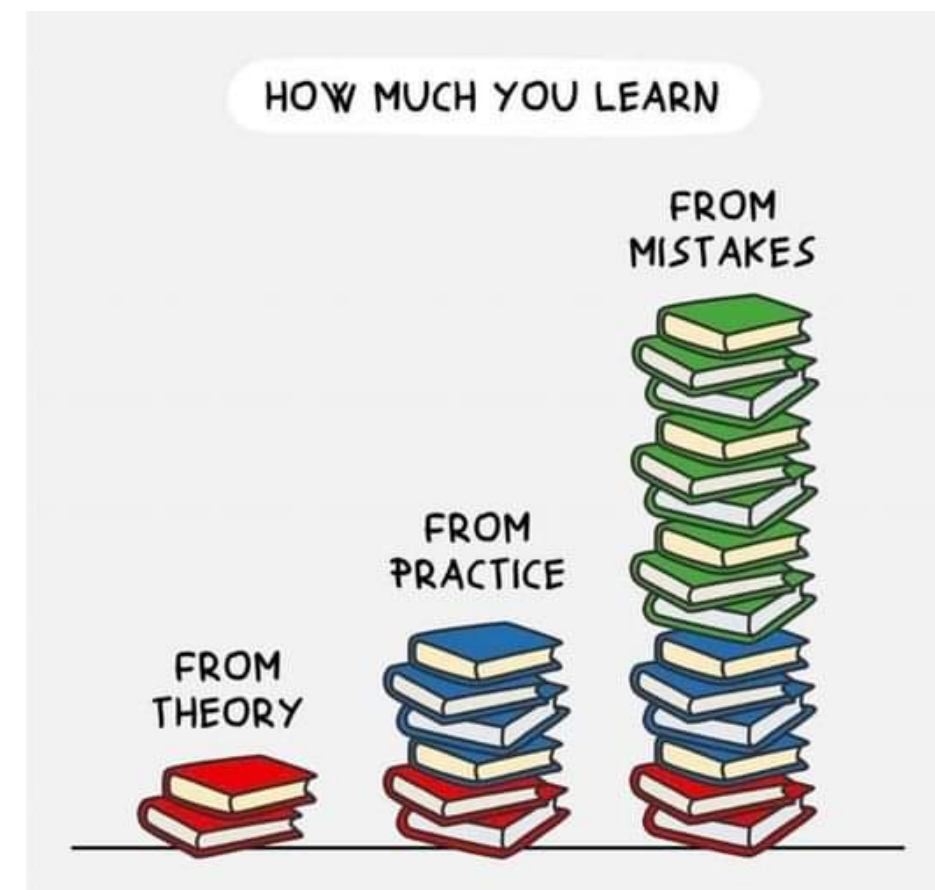
Course Info

- How to prepare for the exam?
 - Interactive attendance to the lectures of prof. Amoroso
 - Self-study of the textbook
 - Interactive attendance to the lab sessions
 - Self-practice at home with homework exercises
 - Group working
 - Attendance to homework sessions



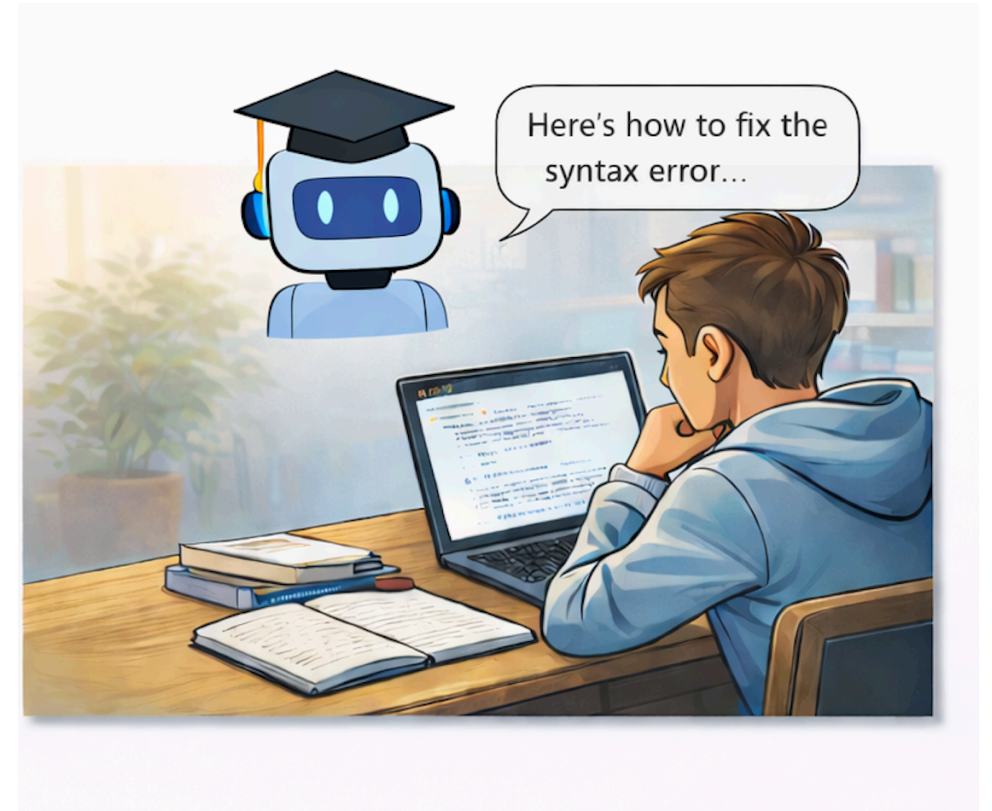
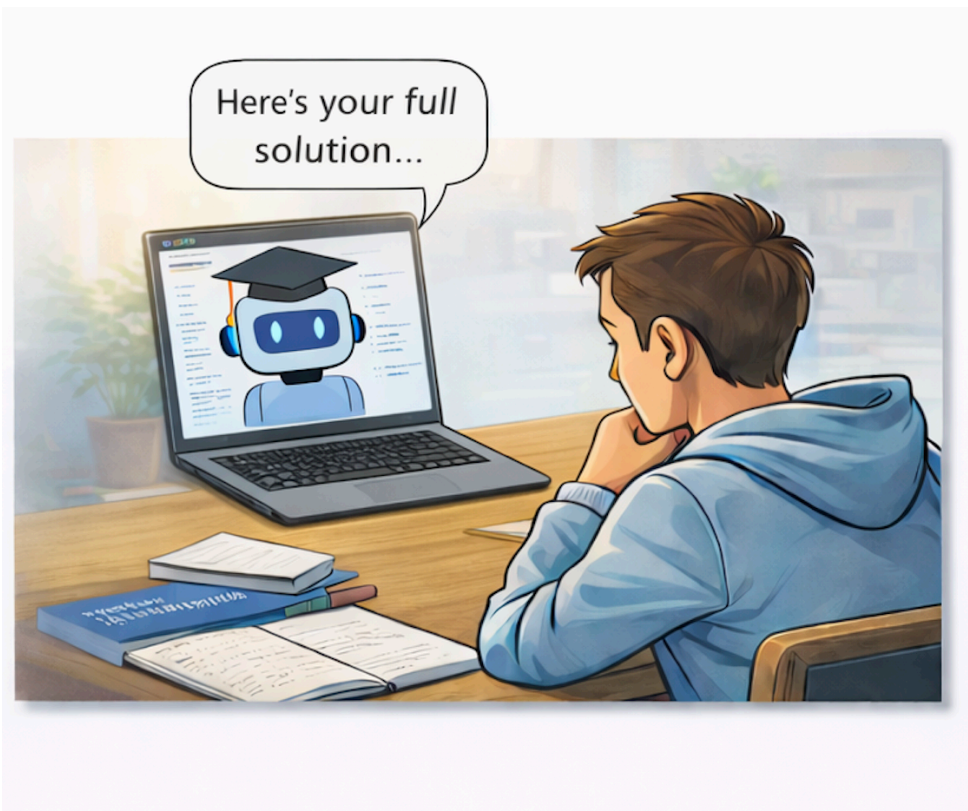
Course Info

- How to prepare for the exam?
 - Interactive attendance to the lectures of prof. Amoroso
 - Self-study of the textbook
 - Interactive attendance to the lab sessions
 - Self-practice at home with homework exercises
 - Group working
 - Attendance to homework sessions



Course Info

- Use of AI for programming?
 - AI could support learning, but should not replace independent learning & coding practice
 - **Do not** use it as a code generator
 - **Use it as a tutor:** get explanations, debug help, hints, code review, extra practice problems



Course Info

- Lab topics **in order**

April

- **Introduction to Python**
- **Branching & input**
- **String slicing & in**
- **For loop & range**
- **While loop**
- Functions
- Tuples
- Lists

May

- Matrices
- Dictionaries
- List, dictionary, tuple comprehension
- String manipulation
- File handling
- Recursive functions
- Basic OOP
- Sample exam

Student Info

- What is your programming background? Vote in Virtuale.
 1. No prior experience
 2. Some experience with Python
 3. Some experience with another programming language
 4. Advanced experience with Python
 5. Advanced experience with another programming language

Integrated Development Environment (IDE)

- A tool that helps to develop code with features like syntax highlighting, autocompletion, smart indentation, running and debugging the code, etc
 - Some examples of IDE for Python: PyCharm, Spyder, IDLE, etc...
- For Python 3 we will use **Thonny**, a really simple IDE specifically designed for teaching
- The application offers just some basic functionalities

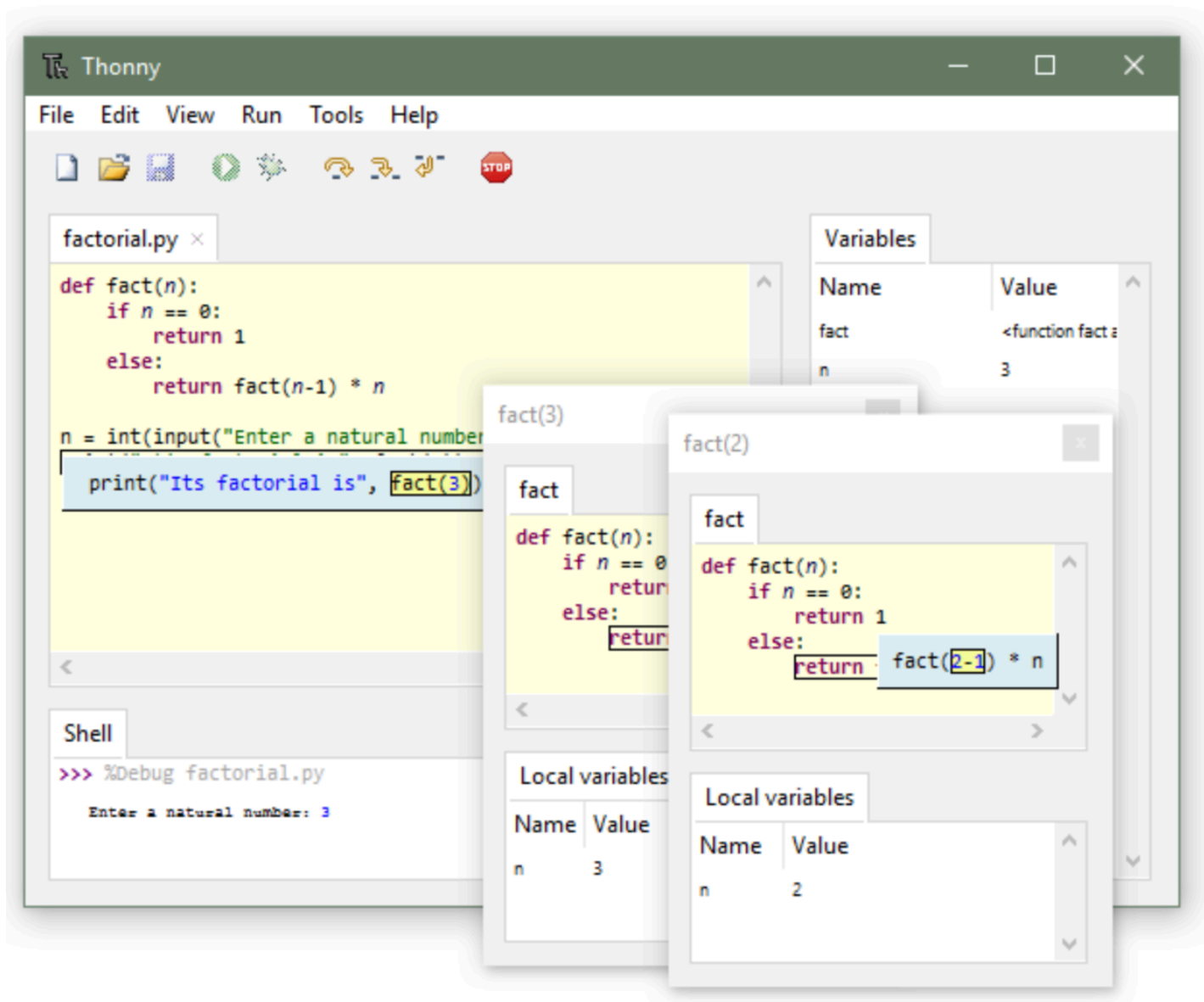
IDE

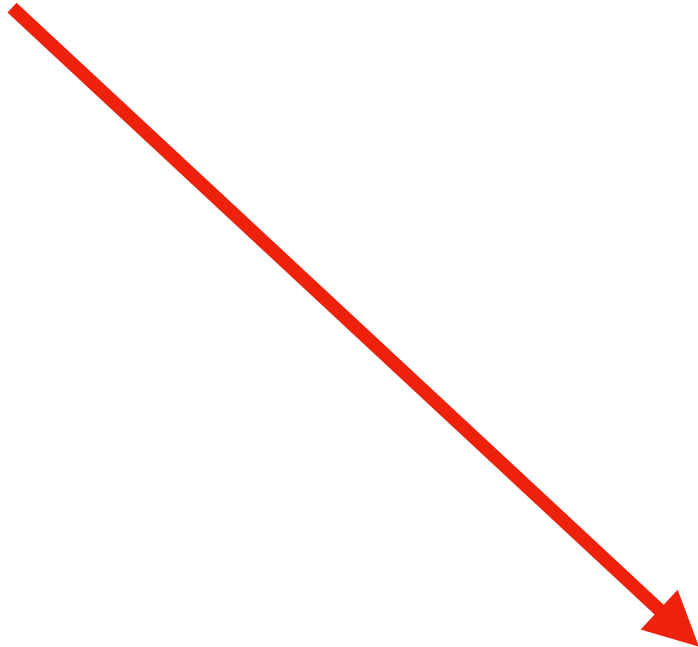
- Already installed in the lab computers
- You can install it on your personal computer by downloading it from thonny.org
- Follow the installation process after running the installer

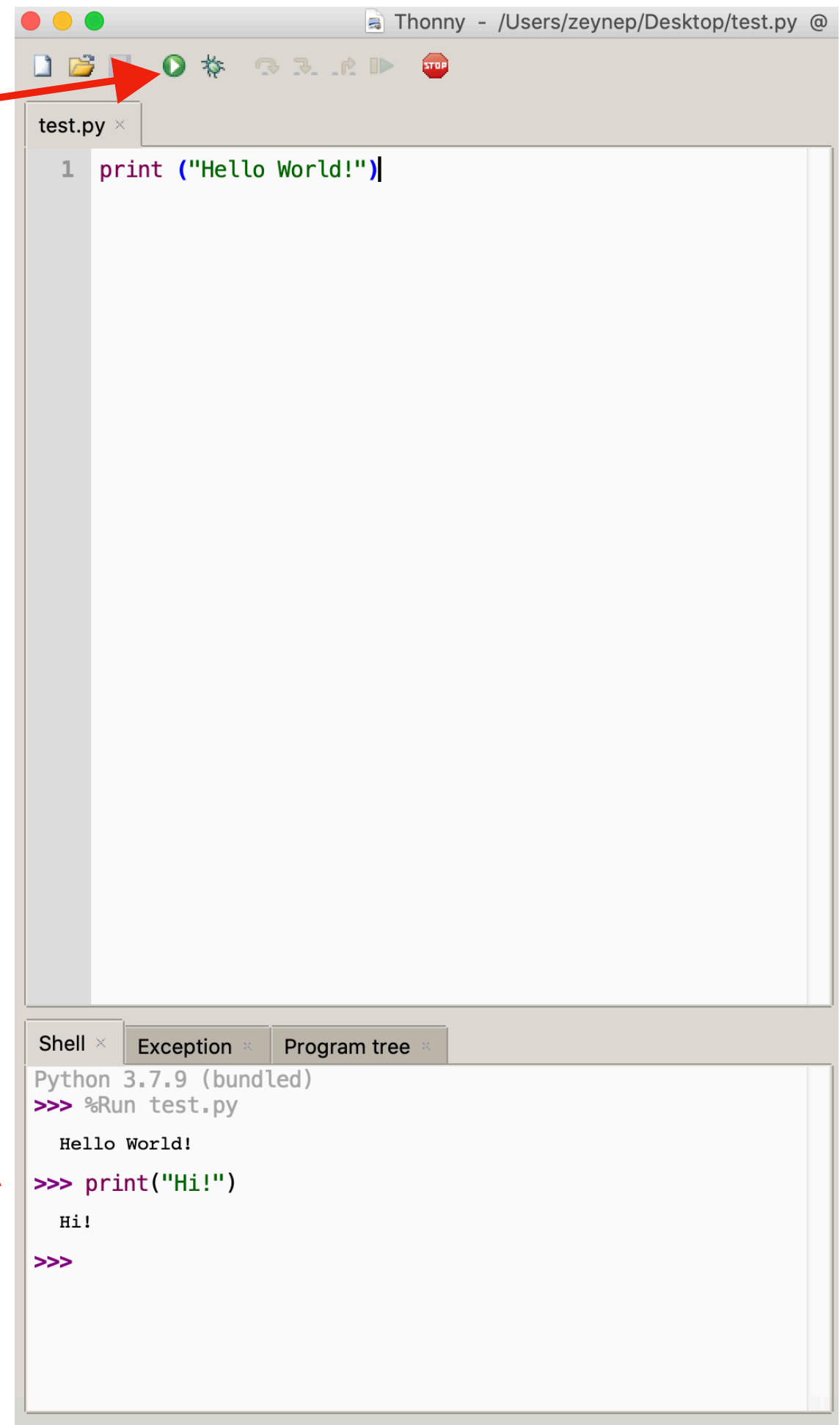
Thonny
Python IDE for beginners



Download version [4.1.7](#) for
[Windows](#) • [Mac](#) • [Linux](#)



- You can write a program in a file and execute  
- You first need to save the Python (.py) file somewhere
- You can run command directly from the shell
“>>>” below and press enter 



Basic Commands/Statements

- We are going to use the `print( " " )` command a lot
 - It is not the best programming practice, but is helpful when learning
 - `print( "Hello World" )`
- Assignment
 - `a = 2` (associates the value 2 with the name a)
 - What is a?

Objects, Expressions and Types

- Objects are the core things that Python programs manipulate
- Every object has a **class** (called **type**) that defines the kinds of things that programs can do with that object
- Types
 - `int`: represent integers (e.g. 3)
 - `float`: represent real numbers (e.g. 3.0)
 - `bool`: represent Boolean values True and False
 - `str`: represent strings (e.g. "Hello")
- Objects and operators defined on them can be combined to form expressions
 - The expression is evaluated to a value (e.g. $3 + 2 = 5$)
 - The value is also an object of a certain type (e.g. $3 + 2$ denotes the object 5 of type `int`)

Objects, Expressions and Types

- Variables provide a way to associate names with objects (e.g. `a`)
- The symbol `=` means assignment
 - `<variable name> = <expression>`
 - Must be read from right to left (e.g. `a = 3+2`)
 - Evaluates the expression on the right of `=` (5 in our example)
 - Associates the evaluated value (5 in our example) to the variable (`a` in our example)
- If `a` already contains a value, it is overwritten and the previous value is lost
- `type(a)` returns the class (type) of the object named by `a`

```
<class 'int'>
```

Objects, Expressions and Types: int and float

- Integer
 - $M = 3$
- Floating point
 - $M = 3.0$
- Expressions
 - $M + M \rightarrow \text{sum (int, float)}$
 - $M * M \rightarrow \text{product (int, float)}$
 - $M / M \rightarrow \text{division, results will be float}$
 - $M // M \rightarrow \text{integer division, results will be integer (int, float)}$
 - $M \% M \rightarrow \text{remainder (only between integer)}$
 - $M ** M \rightarrow \text{power raise (int, float)}$

Objects, Expressions and Types: strings

- $M = \text{"Name"}$
- $N = \text{"Surname"}$
- Expressions
 - $M + N \rightarrow$ concatenates strings M and N (e.g. "NameSurname")
 - $M * 3 \rightarrow$ concatenates 3 times the string M (e.g. "NameNameName")
 - $\text{len}(M) \rightarrow$ returns the length of the string M (e.g. 4 with $M = \text{"Name"}$)
 - $M[i] \rightarrow$ returns the character of M at index i
 - Indexing is from 0 to $\text{len}(M)-1$
 - e.g. $M = \text{"Name"}$, $M[0] \rightarrow \text{'N'}$, $M[3] \rightarrow \text{'e'}$, $M[-1] \rightarrow \text{'e'}$

Objects, Expressions and Types

Try this code and check the two output

```
my_string = "first"  
print(my_string)  
a = "second"  
b = "third"  
my_string = a+b  
print(my_string)
```

Objects, Expressions and Types

Try this code and check the output

```
a = "hello"  
b = 5  
a_by_b = a*b  
print(a_by_b)
```

Objects, Expressions and Types

Try this code and check the output

```
a = 2
b = 3
area = a*b
perimeter = a*2 + b*2
print(area, perimeter)
```

Objects, Expressions and Types

Try this code and check the output

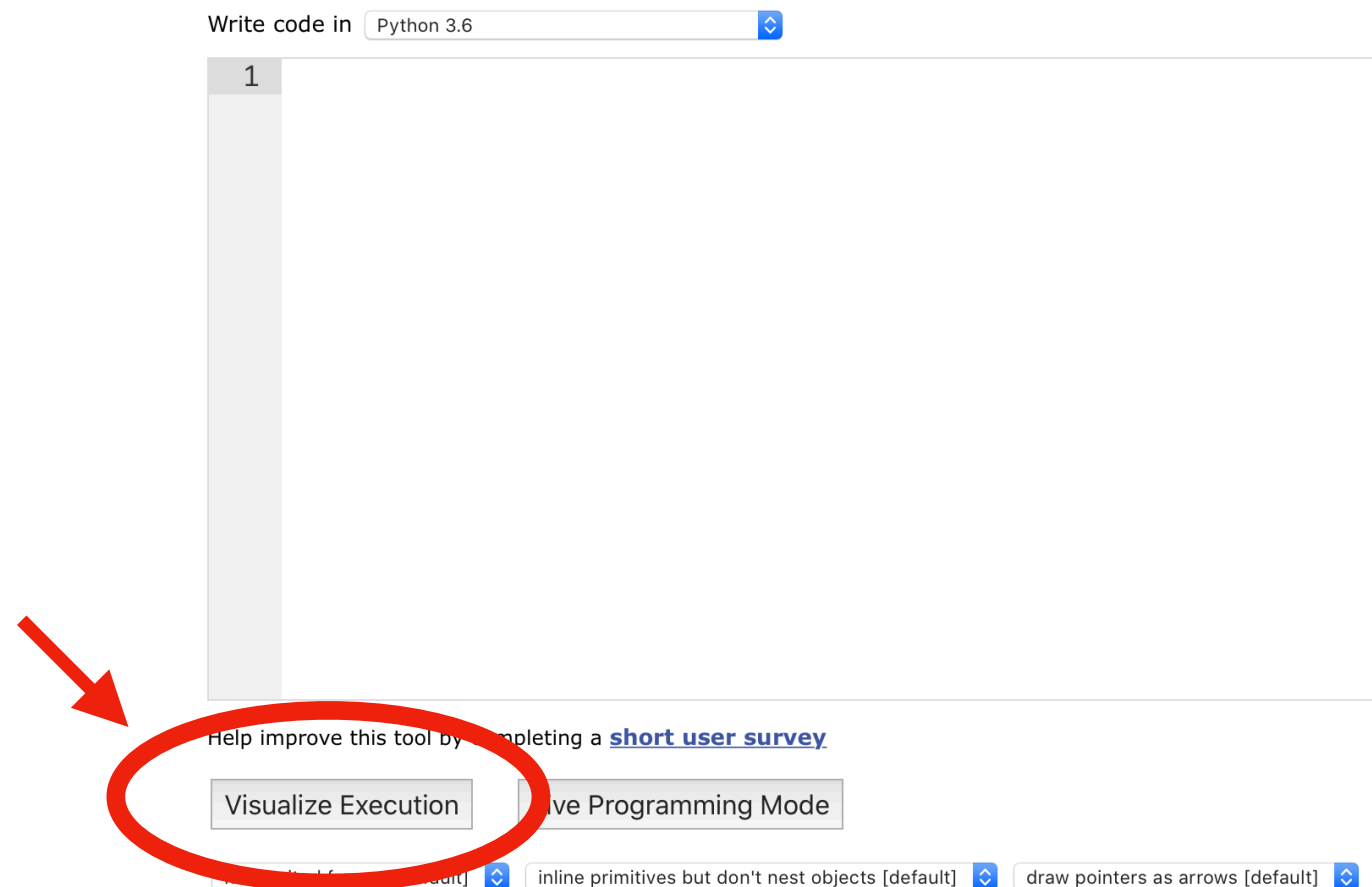
```
a = 42
b = 13
b = a
print(a, b)
```

Tracing

- It is possible to exploit a **tracing tool** to look at the evolution of the state of each variable (instead of doing it by yourself by printing)

<http://pythontutor.com/visualize.html#mode=edit>

- Let's try it!



Tracing

- It is possible to exploit a **tracing tool** to look at the evolution of the state of each variable (instead of doing it by yourself by printing)

<https://pythontutor.com/visualize.html#mode=edit>

- Or just use **Thonny** in **debug mode**

Simple Exercises

1. Write a program that swaps the values associated to two variables; the program must work for every initial value in the two variables (e.g. if $a=7$ and $b=20$, after the execution of the program we should have $a=20$ and $b=7$)
2. Write a program that prints the volume of a sphere with a given radius (e.g. $r=7$), by importing the Pi value from the math module
3. Write a program that prints a given string of four characters in reverse order (e.g. if the string is "ciao", the program will print "oaiC")
4. Given the fact that you can run 10 km in 42 min and 42 sec, write a program that prints your average speed in km/min, km/h, miles/min and miles/h
 - ➡ Calculate how many seconds there are in 42 minutes and 42 seconds
 - ➡ How many miles in a kilometer? (Hint: 1.61 km in a mile)
 - ➡ Your average speed is space/time